الأساسيات — كيف يعمل هذا المشروع، من الصفر

مكتوب لمن هو في سنته الأولى في الإلكترونيات أو النظم المدمجة. لا يفترض معرفة سابقة بناقل SPI، ولا بمشغّلات LED، ولا بخوادم الويب، ولا بأنظمة البناء. كل مصطلح يُشرح عند وروده أول مرة.

إن كنت تعرف ما المسجّل الإزاحي، فانتقل إلى الجزء الرابع.

الجزء ٠ — ما الذي نبنيه فعلاً

لافتة صغيرة تعرض نصاً عربياً على شبكة من مصابيح LED حمراء. تكتب النص على هاتفك، فتظهر الحروف على اللافتة.

ثلاث قطع من العتاد:

```
┌──────────┐        ┌────────┐         ┌──────────┐
│ واي فاي  │ الهاتف │ ESP32  │ مصفوفة  │ أسلاك LED│
│ (متصفّح) │═══════▶│(حاسوب) │════════▶│ (٨ × ٨)  │
└──────────┘        └────────┘         └──────────┘
```

- **الهاتف** حيث تكتب. لا شيء يُثبَّت عليه — إنما يفتح صفحة ويب.
- **الـ ESP32** حاسوب صغير مدمج فيه الواي فاي. ثمنه بضعة دولارات.
- **مصفوفة LED** ٦٤ مصباحاً أحمر مرتّبة في شبكة ٨×٨، وخلفها شريحة قيادة.

بقية هذا المستند تشرح كل قطعة، وكيف يتحدّث بعضها إلى بعض.

الجزء ١ — مخطط التوصيل

خمسة أسلاك تصل الـ ESP32 بوحدة المصابيح. هذا كل شيء.

```
    ESP32                          وحدة MAX7219

┌──────────┐                   ┌──────────────┐
│          │                   │ ●●●●●●●●     │
│ VIN   ───┼──── أحمر ─────────▶│ VCC  ●●●●●   │
│ GND   ───┼──── أسود ─────────▶│ GND (٦٤ مصباحاً)│
│ GPIO13 ──┼──── أخضر ─────────▶│ DIN  ●●●●●   │
│ GPIO14 ──┼──── أزرق ─────────▶│ CLK  ●●●●●   │
│ GPIO27 ──┼──── أصفر ─────────▶│ CS   ●●●●●   │
└──────────┘                   └──────────────┘
```

ماذا يفعل كل سلك

السلك	الاسم الكامل	المهمّة
VCC	جهد التغذية	دخل الطاقة — يزوّد المصابيح بـ ٥ فولت
GND	الأرضي	مسار العودة. كل دارة تحتاجه
DIN	دخل البيانات	يحمل المعلومة نفسها، بتاً واحداً في كل مرة
CLK	النبض	ميقاتية. تقول *الآن* — اقرأ بت البيانات
CS	اختيار الشريحة	تقول *أنا أخاطبك وانتهت الرسالة*

ولمَ سلك نبض أصلاً؟ سلك البيانات لا يحمل إلا جهداً — مرتفعاً أو منخفضاً. فإن أرسلت عشرة بتات قيمتها `1` متتالية، بقي السلك مرتفعاً طوال الوقت. وليس عند المستقبِل ما يميّز أهي ثلاثة بتات أم عشرة. والنبض يحلّ ذلك: مع كل نبضة يأخذ المستقبِل عيّنة واحدة من سلك البيانات. عشر نبضات، عشرة بتات. بلا لبس.

تخيّلها كتهجئة كلمة بصوتك عبر خط هاتف رديء. الحروف هي البيانات، وقولك «التالي… التالي…» هو النبض.

والأرضي ليس اختيارياً. فالجهد دائماً *فرقٌ* بين نقطتين. وقولك «سلك البيانات عند ٣٫٣ فولت» بلا معنى ما لم تتّفق الشريحتان على ما هو الصفر. وسلك الأرضي هو ذلك الاتفاق. وبدونه تتصرف الشاشة عشوائياً بدل أن تفشل فشلاً نظيفاً — وهذا ما يجعله عطلاً مربكاً في تشخيصه.

لمَ ٥ فولت لا ٣٫٣

يعمل الـ ESP32 داخلياً على ٣٫٣ فولت، لكن وحدة المصابيح تريد ٥.

تحتاج المصابيح جهداً معيّناً قبل أن توصّل التيار أصلاً. فالحمراء منها تحتاج نحو ١٫٨ إلى ٢ فولت، وتحتاج شريحة القيادة هامشاً فوق ذلك. وعند ٣٫٣ فولت تخفت المصابيح، والأدقّ من ذلك أن فكرة الشريحة عمّا «يُعدّ منطقاً واحداً» قد لا تعود مطابقة لما يخرجه المتحكم. فتحصل على وميض وتشويش بدل عطل صريح.

وفي لوحة الـ ESP32 قطب **VIN** يمرّر الـ ٥ فولت الآتية من USB، وهو تماماً ما تريده الوحدة.

اختيار قطب واحد له أهمية

يقع CS على **GPIO27** رغم أن **GPIO12** أقرب مادياً إلى بقية الأسلاك.

فـ GPIO12 **قِطب ضبط**. فحين يُشغَّل الـ ESP32 يقرأ الجهد على أقطاب محددة ليقرّر كيف يهيّئ نفسه — وGPIO12 يختار جهد ذاكرة الفلاش الداخلية. ويجب أن يقرأه منخفضاً في تلك اللحظة بالذات. فإن أبقاه سلكٌ مرتفعاً عند التشغيل، اختارت الشريحة جهد الفلاش الخطأ و**رفضت الإقلاع**. تبدو اللوحة ميتة، ولا شيء في العَرَض يشير إلى التوصيل.

درس يُستحسن استيعابه مبكراً: في المتحكمات الدقيقة، لبضعة أقطاب مهامّ خاصة أثناء الإقلاع. راجع ورقة البيانات قبل اختيار الأقطاب لا بعده.

الجزء ٢ — كيف تعمل مصفوفة المصابيح

مشكلة التوصيل

في مصفوفة ٨×٨ أربعة وستون مصباحاً. والتحكم بكل واحد منفرداً يحتاج ٦٤ سلكاً زائد الأرضي. وللوحدة ١٦ قطباً فقط.

والحيلة في طريقة التوصيل:

```
        العمود ٢   العمود ١   العمود ٠   ...
              │          │          │
  الصف ٠ ─────┼─▶├───────┼─▶├───────┼─▶├──
              │          │          │
  الصف ١ ─────┼─▶├───────┼─▶├───────┼─▶├──
              │          │          │
  الصف ٢ ─────┼─▶├───────┼─▶├───────┼─▶├──
```

يجلس كل مصباح عند تقاطع سلك صف مع سلك عمود. ٨ صفوف + ٨ أعمدة = **١٦ سلكاً تتحكم بـ ٦٤ مصباحاً**.

ولا يضيء المصباح إلا حين يسري فيه تيار — وهذا يتطلّب رفع صفّه وخفض عموده. فلإضاءة المصباح عند الصف ٢ والعمود ٥: ارفع الصف ٢ واخفض العمود ٥.

العقبة، والحيلة

لا يمكنك عرض صورة اعتباطية بهذه الطريقة دفعة واحدة. فإن رفعت الصفّين ٠ و١ وخفضت العمودين ٣ و٦، أضأت **أربعة** مصابيح — (٠،٣) و(٠،٦) و(١،٣) و(١،٦) — لا الاثنين اللذين أردتهما. فالصفوف والأعمدة لا تُوجَّه استقلالاً.

والحل **المضاعفة الزمنية**: لا تُضئ أكثر من صف واحد في اللحظة الواحدة.

```
   ← الزمن


┌─────────┬─────────┬─────────┬─────────┬─────────┐
│ … ثم يتكرر │ الصف ٣ │ الصف ٢ │ الصف ١ │ الصف ٠ │
│ أعمدته   │ أعمدته  │ أعمدته  │ أعمدته  │
└─────────┴─────────┴─────────┴─────────┘
  ١ ملي ثا  ١ ملي ثا  ١ ملي ثا  ١ ملي ثا
```

أضئ الصف ٠ بنمط أعمدته الصحيح. أطفئه. أضئ الصف ١ بنمط *أعمدته هو*. وهكذا عبر الثمانية، ثم ابدأ من جديد.

ولا يضيء كل صف إلا ثُمن الوقت — لكن إن تكررت الدورة كاملة أسرع من نحو ٥٠ مرة في الثانية، عجزت عينك عن ملاحقتها. **وبقاء الأثر البصري** يمزج الومضات في صورة ثابتة.

وهو الأثر نفسه الذي يجعل الفيلم يبدو حركةً لا سلسلة صور ساكنة. وكل شاشة LED وكل هاتف وكل تلفاز يعتمد عليه.

وثمن الحيلة: كل مصباح مطفأ سبعة أثمان الوقت، فالشاشة أخفت مما لو عملت كل المصابيح باستمرار. وتعوّض شرائح القيادة ذلك بدفع تيار أكبر خلال فترة الإضاءة القصيرة.

الجزء ٣ — ما المسجّل

هذا هو المفهوم الذي يقوم عليه كل ما عداه.

المسجّل قطعة صغيرة من الذاكرة *داخل الشريحة*. ليس ذاكرة المتحكم العشوائية، بل ذاكرة مبنية داخل الشريحة الطرفية نفسها، وعادةً بعرض بضعة بتات فقط.

والتصوّر الذهني: المسجّل **صفٌّ من المفاتيح تبقى حيث تضعها.**

```
  ٠  ١  ٢  ٣  ٤  ٥  ٦  ٧  البت
 ┌──┬──┬──┬──┬──┬──┬──┬──┐
 │1 │0 │1 │1 │0 │0 │1 │0 │  مسجّل بثمانية بتات
 └──┴──┴──┴──┴──┴──┴──┴──┘
```

اكتب فيه `10110010` فيحتفظ بـ `10110010` حتى تكتب غيره — حتى لو غادرت المكان. وفي مشغّل مصابيح، يتحكّم كل بت عادةً بمصباح واحد: ١ مضاء، ٠ مطفأ.

ولمَ تستخدم الشرائح مسجّلات؟ تحتاج الشريحة أن يُقال لها ماذا تفعل، لكن إضافة قطب لكل إعداد ستجعلها ضخمة. فتعرض بدلاً من ذلك بضعة مسجّلات ووصلة تسلسلية صغيرة واحدة. فيصير الضبط «اكتب هذا الرقم في ذلك المسجّل» بدل «أضف سلكاً آخر».

وللشريحة عادةً عدة مسجّلات، لكلٍّ **رقم** (عنوانه) و**مهمّة**:

العنوان	المهمّة
١	مصابيح الصف ٠ الثمانية
٢	مصابيح الصف ١ الثمانية
…	…
١٠	السطوع
١٢	التشغيل والإطفاء

فـ«اعرض شيئاً ما» تعني: اكتب الرقم الصحيح في المسجّل الصحيح.

الجزء ٤ — المسجّل الشهير ذو الثمانية بتات: 74HC595

إن كنت قد قابلت مسجّلاً إزاحياً في مقرّر دراسي، فهو هذا على الأرجح. وفهمه يجعل MAX7219 بديهياً.

ما يفعله المسجّل الإزاحي

المسجّل الإزاحي يستقبل البتات **واحداً تلو الآخر** (تسلسلياً) ويتيحها **كلها دفعة واحدة** (توازياً).

```
  البيانات ──▶ ┌───┬───┬───┬───┬───┬───┬───┬───┐
              │ب٠ │ب١ │ب٢ │ب٣ │ب٤ │ب٥ │ب٦ │ب٧ │
              └─┬─┴─┬─┴─┬─┴─┬─┴─┬─┴─┬─┴─┬─┴─┬─┘
                ▼   ▼   ▼   ▼   ▼   ▼   ▼   ▼
               ثمانية أقطاب خرج
```

تدفع كل نبضة البت الداخل من طرف، فينزاح كل ما بالداخل موضعاً واحداً — كأناس يتزحزحون على مقعد ليجلس قادم في طرفه. وبعد ثماني نبضات تصطفّ بتاتك الثمانية، فتنسخها نبضة **مزلاج** إلى أقطاب الخرج دفعة واحدة.

ولمَ المزلاج مهم: بدونه سترتجف الأقطاب عبر قيم لا معنى لها أثناء الإزاحة. فالمزلاج يعني أن الخرج لا يتغيّر إلا حين تقول أنت إن البيانات اكتملت.

ولمَ هذا مفيد: ثلاثة أقطاب من المتحكم (بيانات، نبض، مزلاج) تصير ثمانية مخارج. وبربط شريحتين تصير الثلاثة ستة عشر — إذ تُغذّي البتات الخارجة من الأولى الشريحةَ الثانية ببساطة.

قيادة المصابيح بـ 74HC595 — وما يجب أن تضيفه

الـ 74HC595 شريحة *عامة الغرض*. لا تعرف شيئاً عن المصابيح. ولبناء شاشة ٨×٨ بها عليك أن توفّر كل ما عداها بنفسك:

ما عليك إضافته	لماذا
مقاومة لكل مصباح (٨ فأكثر)	الشريحة لا تحدّ التيار؛ وبدون مقاومات تحترق المصابيح
شريحة ثانية للصفوف	واحدة تقود ٨ أعمدة؛ وتحتاج الصفوف مشغّلها الخاص
ترانزستورات	صفٌّ كامل من المصابيح يسحب تياراً يفوق ما تعطيه الشريحة
شفرة المضاعفة	*برنامجك أنت* عليه أن يدوّر الصفوف بسرعة كافية، إلى الأبد
منطق السطوع	أي تعتيم تريده، تكتبه بنفسك

وحلقة المضاعفة تلك هي العبء الحقيقي. إذ يجب أن تعمل بلا توقف — فتأخّر بضعة أجزاء من الألف من الثانية في مكان آخر من برنامجك يظهر وميضاً مرئياً. فتصير شفرتك مسؤولة دائماً عن تحديث الشاشة، فوق كل ما يُفترض بها فعله.

الجزء ٥ — MAX7219: الفكرة نفسها، متخصّصة

الـ MAX7219 مسجّل إزاحي **زائد كل ما كان عليك إضافته.**

```
  74HC595                      MAX7219
  ─────────                    ───────
  مسجّل إزاحي ◀──────           مسجّل إزاحي
  (تضيف مقاومات)       ──────▶ مشغّلات تيار ثابت مدمجة
  (تضيف مشغّل صفوف)    ──────▶ مشغّل صفوف مدمج
  (تكتب المضاعفة)      ──────▶ مضاعفة في العتاد
  (تكتب التعتيم)       ──────▶ ١٦ مستوى سطوع مدمجة
  (تتابع الحالة)       ──────▶ ٨ مسجّلات تحفظ الصورة
```

ما يفعله عنك

تيار ثابت، بمقاومة واحدة إجمالاً. مقاومة واحدة تضبط التيار لكل المصابيح الأربعة والستين. لا ٦٤ مقاومة — واحدة. ويحصل كل مصباح على السطوع نفسه مهما كان عدد المضاء منها، وهذا ما لا يقدر عليه الـ 595.

المضاعفة في السيليكون. تدوّر الشريحة الصفوف بنفسها، باستمرار، إلى الأبد. فيتحرّر برنامجك لحظة إرسال البيانات. **وهذه هي النقطة الكبرى** — إذ يختفي عبء التحديث كله.

ذاكرة للصورة. ثمانية مسجّلات تحفظ الصورة الحالية. اكتب مرة، وتبقى الشريحة تعرضها. لا شيء تتعهّده.

التحكم بالسطوع. اكتب رقماً من ٠ إلى ١٥ في مسجّل السطوع. بلا دارات إضافية.

المقارنة

	74HC595	MAX7219
الغرض	مسجّل إزاحي عام	مشغّل مصفوفة مصابيح
المقاومات	واحدة لكل مصباح	واحدة إجمالاً
المضاعفة	شفرتك، إلى الأبد	عتاد
السطوع	دارتك	مدمج، ١٦ مستوى
يحفظ الصورة	لا	نعم
شرائح لمصفوفة ٨×٨	٢ فأكثر مع ترانزستورات	١
الكلفة	أرخص للشريحة	أرخص إجمالاً لهذه المهمّة
المرونة	يقود أي شيء	مصابيح فقط

فأيّهما تستخدم؟ الـ 595 حين تحتاج مخارج عامة — مرحّلات، شرائح أخرى، أحمال مختلطة. والـ MAX7219 حين تقود مصفوفات مصابيح، لأنه يزيل صنفاً كاملاً من العمل.

والـ 595 شريحة *تعليمية* أفضل لأنه يجبرك على أداء العمل بيدك. والـ MAX7219 شريحة *بناء* أفضل لأنك لست مضطراً.

الجزء ٦ — البروتوكول: SPI

تُحمَّل الشريحتان بالطريقة نفسها، باستخدام **SPI** — الواجهة الطرفية التسلسلية.

و«تسلسلي» يعني أن البتات تسافر **واحداً بعد آخر على سلك واحد**، بدل ثمانية أسلاك متجاورة.

الإشارات الثلاث

```
ESP32 (سيّد)                      MAX7219 (تابع)
    CLK  ─────┐ ┌┐ ┌┐ ┌┐ ┌┐ ┌───────▶ CLK
              └─┘└─┘└─┘└─┘└─┘
    DIN  ────── 1   0   1   1 ────────▶ DIN
    CS   ───╲_____________________╱─── CS
             └──── الرسالة ──────┘
```

- **CLK** — يولّد السيّد النبضات. ومع كل نبضة يأخذ التابع عيّنة من خط البيانات.
- **DIN** — بت واحد لكل نبضة.
- **CS** — يبقى منخفضاً طوال الرسالة. وحافّته **الصاعدة** في النهاية تعني *اكتملت الرسالة، نفّذها*.

والـ CS هو المزلاج من الجزء الرابع، باسم آخر. فالـ MAX7219 يجمع البتات ما دام CS منخفضاً، ويطبّقها حين يرتفع.

السيّد والتابع: يولّد السيّد النبض ويبدأ كل تبادل. ولا يفعل التابع إلا الاستجابة. وهنا الـ ESP32 سيّد والـ MAX7219 تابع — ولهذا لا يستطيع الـ MAX7219 أن يبادر بمعلومة أبداً. فليس له **سلك خرج إطلاقاً**.

ولتلك الحقيقة أثر عملي في هذا المشروع: لا يمكنك أن تسأل الوحدة كيف وُصِّلت داخلياً. فلا مسار للجواب. وكل ما تحتاج معرفته عنها يُكتشف بتجربة شيء *والنظر إلى اللوحة*.

ما يتوقّعه الـ MAX7219

ستة عشر بتاً لكل رسالة:

```
┌──────────┬──────────┬────────────────┐
│ ٨ بتات   │ ٤ بتات   │ ٤ بتات          │
│ بيانات   │ عنوان    │ مهمَلة          │
│          │ المسجّل  │                │
└──────────┴──────────┴────────────────┘
```

فإرسال `0x01, 0xFF` يعني *المسجّل ١ (الصف ٠)، وكل البتات الثمانية مضاءة* — فيضيء الصف العلوي كاملاً.

والـ 74HC595 ليس فيه حقل عنوان أصلاً؛ إنما تدفع ٨ بتات وتُزلِج. أما الـ MAX7219 فينفق ٤ بتات إضافية على عنوان لتحفظ شريحة واحدة عدة مسجّلات مستقلة. وذلك هو الفرق بين قطعة عامة وأخرى متخصّصة، ظاهراً في صيغة الاتصال نفسها.

الربط المتسلسل

تُربط الشريحتان بالطريقة نفسها: البتات الخارجة من الشريحة ١ تسري إلى الشريحة ٢.

ونتيجةٌ تفاجئ الجميع: الكلمة التي ترسلها **أولاً** ينتهي بها المطاف في **آخر** شريحة في السلسلة. فكل ما ترسله عليه أن يعبر الشرائح السابقة ليصل إلى اللاحقة، فيدفع البيانات السابقة أمامه. ولذلك حين تكتب شفرة لسلسلة، ترسل البيانات بترتيب معكوس.

الجزء ٧ — كيف يستطيع ESP32 تقديم صفحة ويب؟

هذا هو الجزء الذي يبدو سحرياً وليس كذلك.

ما خادم الويب فعلاً

خادم الويب مجرّد **برنامج ينتظر طلباً ويرسل نصاً في المقابل**. هذه هي المهمّة كلها. ولا شيء فيها يستلزم حاسوباً كبيراً.

حين تفتح `http://192.168.4.1/` يقوم متصفّحك بـ:

1. فتح اتصال بذلك العنوان
2. إرسال رسالة نصية قصيرة: `GET / HTTP/1.1`
3. الانتظار
4. استقبال نصّ — HTML — ورسمه

وجانب الخادم هو: أنصت، استقبل `GET /`، أعِد الـ HTML. بضع مئات من الأسطر، وفي الـ ESP32 متسع وافر.

الطبقات المعنيّة

```
┌──────────────────────────────────┐
│ صفحتك HTML  (مخزّنة في الفلاش)    │ ← أنت تكتبها
├──────────────────────────────────┤
│ خادم HTTP  — يفهم GET وPOST       │ ← مكتبة
├──────────────────────────────────┤
│ حزمة TCP/IP — مكتبة ← │      عناوين وحزم
├──────────────────────────────────┤
│ شريحة ومكتبة ← │  802.11  مشغّل الواي فاي — الراديو،
├──────────────────────────────────┤
│ سيليكون ← │           عتاد راديو الواي فاي
└──────────────────────────────────┘
```

أنت تكتب الطبقة العليا. وكل ما تحتها يأتي مع حزمة التطوير.

وضع نقطة الوصول — لا حاجة إلى موجّه

تعمل شرائح الواي فاي في وضعين:

- **محطّة (STA)** — تنضمّ إلى شبكة قائمة. وهو ما يفعله هاتفك عادةً.
- **نقطة وصول (AP)** — *تصير* شبكةً ينضمّ إليها غيرها. وهو ما يفعله الموجّه.

ويستطيع الـ ESP32 الوضعين. وهنا يعمل **نقطةَ وصول**، فينشئ شبكة واي فاي خاصة به ينضمّ إليها هاتفك مباشرة. **لا موجّه، ولا إنترنت، ولا شريحة اتصال.** فتعمل اللافتة قائمةً بذاتها في أي محل.

كيف يحصل الهاتف على عنوان

حين ينضمّ هاتفك يحتاج عنوان IP. ويشغّل الـ ESP32 **خادم DHCP** صغيراً — الخدمة نفسها التي يشغّلها موجّه بيتك — فيمنحه عنواناً، ويخبره أن اللافتة نفسها على `192.168.4.1`.

ويخبر الـ DHCP الهاتفَ أيضاً بمعلومات مفيدة، منها أي خادم يستخدم للبحث عن الأسماء. وهذه التفصيلة أهمّ مما تبدو: فإن غابت، أمكن للهاتف أن ينضمّ بنجاح ويتصرّف مع ذلك كأن الشبكة معطّلة.

أين تُخزَّن الصفحة

يُدمَج ملف الـ HTML **داخل البرنامج الثابت** ويقيم في ذاكرة الفلاش إلى جانب البرنامج. فلا بطاقة ذاكرة ولا نظام ملفات يُدار. وحين يصل طلب، يرسل الخادم تلك البايتات من الفلاش مباشرة.

الجزء ٨ — أدوات البناء: أردوينو مقابل PlatformIO مقابل ESP-IDF

يقع الخلط بين هذه الثلاثة باستمرار، ويعود بعض ذلك إلى أنها ليست من *الصنف* نفسه.

التوضيح الذي يحلّ أكثر اللبس

- **أردوينو إطار عمل** — مجموعة من الدوال المبسّطة (`digitalWrite`، `setup`، `loop`).
- **ESP-IDF إطار عمل** — حزمة التطوير الرسمية من إسبريسيف للـ ESP32.
- **PlatformIO أداة بناء** — وليست إطاراً منافساً. تُصرّف وترفع المشاريع، وتستطيع بناء **أيٍّ** من الإطارين السابقين.

فـ«أردوينو مقابل ESP-IDF» اختيار حقيقي. أما «PlatformIO مقابل ESP-IDF» فخطأ في التصنيف — إذ يستطيع PlatformIO بناء مشاريع ESP-IDF.

أردوينو

```
void setup() { pinMode(13, OUTPUT); }
void loop()  { digitalWrite(13, HIGH); delay(500); }
```

الجيّد: أسهل بداية ممكنة، ومنظومة مكتبات ضخمة، ويعمل عبر لوحات كثيرة.

الحدود: يخفي العتاد، وهذا يصير مشكلة حين تحتاج شيئاً لم يتوقّعه. كُتب أصلاً لشرائح ثمانية البتات، فالميزات الخاصة بالـ ESP32 — النواتان، ومجدول المهام، والضبط العميق — إما مرهقة أو غائبة.

ESP-IDF

حزمة إسبريسيف نفسها — التي بُني عليها أردوينو-للـ-ESP32 ذاته.

```
void app_main(void) {
    gpio_set_direction(GPIO_NUM_13, GPIO_MODE_OUTPUT);
    gpio_set_level(GPIO_NUM_13, 1);
}
```

الجيّد: وصول كامل إلى كل ميزة. ويتضمّن **FreeRTOS**، وهو نظام تشغيل زمن حقيقي، فتستطيع تشغيل عدة مهام متزامنة — واحدة تحرّك الشاشة وأخرى تخدم صفحات الويب، دون تشبيك يدوي. والضبط عبر `menuconfig` بدل تعديل الشفرة.

الحدود: أكثر تعلّماً، وأكثر إعداداً، ومفاهيم أكثر قبل أن يومض أول مصباح.

PlatformIO

امتداد لـ VS Code يدير أدوات البناء والمكتبات. وتختار الإطار في ملف ضبط:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf      ; أو "arduino"
```

الجيّد: تكامل مريح مع المحرّر، وإدارة أنيقة للاعتماديات.

الحدود: طبقة إضافية بينك وبين الأدوات الرسمية، قد تتأخّر عنها أو تحجب أخطاءها.

لمَ يستخدم هذا المشروع ESP-IDF

المتطلّب	لمَ ESP-IDF
نقطة وصول + DHCP + خادم HTTP	كلها متضمَّنة ومتكاملة على نحو سليم
تحريك الشاشة *أثناء* تقديم الصفحات	مهام FreeRTOS تجعل ذلك مباشراً
تغيير الأقطاب دون تعديل الشفرة	`menuconfig`
تحكّم دقيق في SPI والتوقيت	وصول مباشر إلى المشغّلات

مع أردوينو معظم هذا ممكن لكن بمكابدة. ومع ESP-IDF هو ما صُنعت الحزمة *من أجله*.

الجزء ٩ — تثبيت ESP-IDF

```
# ١. المتطلبات (أوبونتو/ديبيان)
sudo apt install git wget flex bison gperf python3 python3-pip \
                 python3-venv cmake ninja-build ccache libffi-dev \
                 libssl-dev dfu-util libusb-1.0-0

# ٢. تنزيل الحزمة
mkdir -p ~/esp && cd ~/esp
git clone -b v5.3.2 --recursive https://github.com/espressif/esp-idf.git

# ٣. تثبيت المصرّف للـ ESP32
cd ~/esp/esp-idf
./install.sh esp32

# ٤. التفعيل — مطلوب في كل نافذة طرفية جديدة
. ./export.sh
```

ولمَ `--recursive`؟ تتضمّن ESP-IDF مشاريع أخرى (FreeRTOS، وlwIP، وmbedTLS) كوحدات فرعية. وبدون ذلك الخيار تنزّل حزمة ناقصة، فيفشل البناء فشلاً مربكاً.

ولمَ `export.sh` في كل مرة؟ يضيف المصرّف و`idf.py` إلى مسار البحث لتلك النافذة فقط. وهو لا يثبّت عمومياً عمداً، لتُبقي عدة إصدارات جنباً إلى جنب. ونسيانه يعطي `idf.py: command not found` — وهذا كل ما يعنيه.

صلاحية المنفذ التسلسلي

```
ls -l /dev/ttyUSB0           # هل تظهر اللوحة؟
groups | grep dialout        # هل يُسمح لك باستخدامها؟
sudo usermod -aG dialout $USER  # إن لم يكن — ثم سجّل الخروج والدخول
```

في لينكس تنتمي المنافذ التسلسلية إلى مجموعة `dialout`. وبدون العضوية تحصل على «رُفض الإذن»، وهو ما يبدو كلوحة معطوبة وليس كذلك.

ولا تضغط زراً للرفع. فشريحة USB التسلسلية على اللوحة تستخدم خطّي تحكّم لإعادة ضبط الـ ESP32 وإدخاله وضع الإقلاع تلقائياً. وأداة الرفع تتولّى ذلك.

الجزء ١٠ — استخدام ESP-IDF

```
. ~/esp/esp-idf/export.sh      # مرة لكل نافذة طرفية
cd firmware

idf.py set-target esp32        # مرة لكل مشروع
idf.py menuconfig              # تغيير الإعدادات — الأسهم، Enter، S للحفظ، Q للخروج
idf.py build                   # التصريف
idf.py -p /dev/ttyUSB0 flash   # الرفع
idf.py -p /dev/ttyUSB0 monitor # متابعة رسائل اللوحة (للخروج [-Ctrl)

idf.py -p /dev/ttyUSB0 flash monitor   # الأمر اليومي
```

ما هو menuconfig

قائمة إعدادات مُصرَّفة داخل بنائك. فبدل تعديل ثوابت في ملفات المصدر، تُعلَن الخيارات في ملف اسمه `Kconfig`، ويعرضها `menuconfig` قائمةً.

ويُعلن هذا المشروع أموراً كأيّ أقطاب تستخدمها الشاشة، وكم وحدة موصولة، واسم شبكة الواي فاي. فتغيير التوصيل يعني تغيير مُدخَل في قائمة، لا تنقيباً في الشفرة.

ملفات ستقابلها

الملف	الغرض
`CMakeLists.txt`	يعدّد ملفات المصدر — وصفة البناء
`Kconfig.projbuild`	يعلن خياراتك في `menuconfig`
`sdkconfig.defaults`	قيم البداية للإعدادات، ويصحّ إيداعها في المستودع
`sdkconfig`	الإعدادات الحالية **المولَّدة** — لا تودعها
`build/`	خرج المصرّف — لا تودعه

الجزء ١١ — كيف تُنظَّم الشفرة

ليس ماذا يفعل كل سطر، بل كيف تتلاءم القطع. وهذه البنية مشتركة بين معظم مشاريع لغة C.

ثلاثة أصناف من الأشياء

١. التضمينات — جلب شفرة كتبها غيرك

```
#include <string.h>            // مكتبة C القياسية
#include "driver/spi_master.h"  // مشغّل SPI في ESP-IDF
#include "max7219.h"           // وحدتنا نحن
```

`#include` تعني *الصق تصريحات ذلك الملف هنا*. والأقواس الزاوية تعني مكتبة نظام، والاقتباسات تعني ملفاً في هذا المشروع.

٢. الدوال — كتل عمل مسمّاة

```
void fb_set_pixel(framebuffer_t *fb, int x, int y, bool on)
{
    ...
}
```

تُقرأ هكذا: *لا تعيد شيئاً* (`void`)، *واسمها* `fb_set_pixel`، *وتأخذ* ذاكرة إطار، وسينياً، وصادياً، وحالة إضاءة.

٣. الوحدات — ملفات مجمّعة بحسب المسؤولية

الوحدة **زوج** من الملفات:

- `max7219.h` — **الترويسة**: ما تقدّمه هذه الوحدة. قائمة الطعام.
- `max7219.c` — **التنفيذ**: كيف تفعله فعلاً. المطبخ.

وتُضمِّن الملفات الأخرى الترويسة وتستدعي الدوال. ولا تحتاج قط إلى قراءة ملف الـ `.c`.

ولمَ نفصلهما؟ لتستطيع تغيير *كيفية* عمل شيء دون أن يعبأ أحد، ما دامت قائمة الطعام كما هي. وهذه أنفع فكرة مفردة في هيكلة البرمجيات، وهي ما يجعل المشروع قابلاً للبقاء وهو ينمو.

وحدات هذا المشروع

الوحدة	المسؤولية
`main.c`	الإقلاع والتنسيق
`max7219`	مخاطبة شريحة قيادة المصابيح
`framebuffer`	حفظ الصورة في الذاكرة
`frame_rx`	استقبال الصور عبر الكبل
`wifi_ap`	إنشاء شبكة الواي فاي
`http_ui`	تقديم صفحة الويب
`font5x7 / text5x7`	رسم الحروف اللاتينية، للاختبار
`www/index.html`	الصفحة التي يفتحها هاتفك

ولكلٍّ مهمّة **واحدة**. فـ `max7219.c` يعرف الشريحة ولا يعرف شيئاً عن الواي فاي. و`wifi_ap.c` يعرف الشبكات ولا يعرف شيئاً عن المصابيح. وهذا الفصل مقصود — إذ يتيح لك فهم ملف واحد دون استحضار المشروع كله في ذهنك.

أين يبدأ البرنامج

تبدأ برامج C المعتادة عند `main()`. أما برامج ESP-IDF فتبدأ عند **`app_main()`**، لأن نظام التشغيل يعمل أولاً ثم يستدعي شفرتك.

```
void app_main(void)
{
    max7219_init(...);      // ١. أيقظ الشاشة
    fb_init(...);           // ٢. هيّئ مكاناً لصورة
    wifi_ap_start(...);     // ٣. أنشئ شبكة الواي فاي
    http_ui_start(...);     // ٤. ابدأ تقديم الصفحة
    // … ثم يواصل البرنامج عمله، معالجاً الأحداث
}
```

لاحظ أن `app_main` لا تدور إلى الأبد. فتحت **FreeRTOS** يجري العمل في **مهام** مستقلة — واحدة تحدّث الشاشة وأخرى تعالج طلبات الويب — يتنقّل نظام التشغيل بينها. و`app_main` تهيّئها ثم تنتهي.

وهذا بالضبط ما كان سيؤلم في حلقة `loop()` الوحيدة في أردوينو، وهو السبب الرئيس لاستخدام هذا المشروع ESP-IDF.

الجزء ١٢ — ضمّ السلسلة كلها

```
 ١. تكتب العربية على هاتفك
            ↓
 ٢. تحوّل الصفحة الحروف إلى شبكة من نقاط مضاءة ومطفأة
            ↓
 ٣. تُحزَم النقاط في بايتات — ٨ نقاط لكل بايت
            ↓
 ٤. تُرسَل عبر الواي فاي إلى الـ ESP32
            ↓
 ٥. يخزّنها في ذاكرة الإطار
            ↓
 ٦. يرسلها عبر SPI إلى مسجّلات الـ MAX7219
            ↓
 ٧. يضاعف الـ MAX7219 الصفوف باستمرار
            ↓
 ٨. تمزج عينك الومضات في حروف ثابتة
```

وكل خطوة في تلك السلسلة قد شُرحت الآن: ما المسجّل (٣)، وكيف تضاعف المصفوفة (٢)، وأي بروتوكول يحمل البتات (٦)، ولمَ يستطيع الـ ESP32 استضافة صفحة (٧)، وكيف رُتّبت الشفرة لتفعل ذلك (١١).

مسرد المصطلحات

المصطلح	المعنى
نقطة وصول (AP)	جهاز ينشئ شبكة واي فاي لينضمّ إليها غيره
بت / بايت	صفر أو واحد مفرد؛ وثمانية بتات معاً
اختيار الشريحة (CS)	إشارة تعني «أنا أخاطبك» / «انتهت الرسالة»
DHCP	خدمة توزّع عناوين IP على الأجهزة المنضمّة
البرنامج الثابت	برمجية مخزَّنة داخل الجهاز لا على قرص
ذاكرة الفلاش	تخزين يبقى بعد انقطاع الطاقة؛ يحمل البرنامج
FreeRTOS	نظام تشغيل صغير يتيح عدة مهام متزامنة
GPIO	قطب دخل/خرج عام الغرض يتحكم به برنامجك
مزلاج	ينسخ البتات المجمّعة إلى المخارج دفعة واحدة
مضاعفة زمنية	مشاركة الأسلاك باستخدامها لأشياء مختلفة بالتناوب
بقاء الأثر البصري	مزج العين للومضات السريعة في صورة ثابتة
مسجّل	ذاكرة صغيرة داخل شريحة تحفظ إعداداً أو بيانات
تسلسلي	إرسال البتات واحداً بعد آخر على سلك واحد
مسجّل إزاحي	شريحة تستقبل البتات فرادى وتُخرجها مجتمعة
SPI	بروتوكول تسلسلي يستخدم خطوط نبض وبيانات واختيار
قِطب ضبط	قطب يُقرأ عند التشغيل لتهيئة الشريحة
أدوات البناء	المصرّف والأدوات التي تحوّل الشفرة إلى برنامج ثابت